# GELİŞTİRME ORTAMI,KOD STANDARTLARI VE VERSİYON KONTROL SİSTEMİ

## 1. Giriş

Yazılım geliştirme sürecine başlamadan önce uygun bir geliştirme ortamının hazırlanması büyük önem taşımaktadır. Geliştirme ortamı; programlama dili, kod editörü, gerekli kütüphaneler ve versiyon kontrol sistemlerinden oluşmaktadır. Bu çalışmada geliştirme ortamı hazırlanmış, gerekli yazılımlar kurulmuş ve kodlama standartları ile versiyon kontrol sistemi yapılandırılmıştır.

Bu proje kapsamında programlama dili olarak **Python**, kod editörü olarak **Visual Studio Code** ve versiyon kontrol sistemi olarak **Git** kullanılmıştır.

## 2. Gerekli Yazılımların Kurulması

Geliştirme ortamının oluşturulması için öncelikle gerekli yazılımlar sisteme kurulmuştur. Programlama dili olarak Python kurulmuş ve bu kurulum ile birlikte Python paket yöneticisi olan pip sisteme dahil edilmiştir. Pip, projede kullanılacak olan kütüphanelerin kolay bir şekilde kurulmasını sağlamaktadır.

Kod yazma ve düzenleme işlemlerinin daha verimli yapılabilmesi için Visual Studio Code editörü tercih edilmiştir. Visual Studio Code, eklenti desteği sayesinde farklı programlama dilleri ile uyumlu çalışabilen güçlü bir geliştirme ortamı sunmaktadır.

Proje dosyalarının sürüm kontrolünü sağlamak amacıyla Git versiyon kontrol sistemi kurulmuştur. Git sayesinde proje üzerinde yapılan değişiklikler takip edilebilmekte ve gerektiğinde önceki sürümlere geri dönülebilmektedir.

Kurulum işlemlerinin doğrulanması amacıyla terminal üzerinden aşağıdaki komutlar çalıştırılmıştır:

**python --version**
**pip --version**
**git --version**

## 3. Proje Klasörünün Oluşturulması

Proje dosyalarının düzenli bir şekilde saklanabilmesi için bir proje klasörü oluşturulmuştur. Bu klasör içerisinde proje ile ilgili tüm kaynak dosyaları ve yapılandırma dosyaları yer almaktadır.

Terminal üzerinden aşağıdaki komutlar kullanılarak proje klasörü oluşturulmuştur:

**mkdir proje**
**cd proje**

Bu klasör yazılım geliştirme sürecinde ana çalışma dizini olarak kullanılmaktadır.

**4. Sanal Ortamın Oluşturulması**

Python projelerinde bağımlılıkların yönetilmesi amacıyla sanal ortam kullanılması önerilmektedir. Sanal ortam sayesinde projeye ait kütüphaneler sistemdeki diğer projelerden bağımsız olarak yönetilebilmektedir.

Sanal ortam oluşturmak için aşağıdaki komut kullanılmıştır:

**python -m venv venv**

Sanal ortamın aktif edilmesi için aşağıdaki komutlar çalıştırılmıştır:

Windows işletim sistemi için:

**venv\Scripts\activate**

Linux veya macOS işletim sistemi için:

**source venv/bin/activate**

**5. Gerekli Kütüphanelerin Kurulması**

Proje kapsamında kullanılacak olan kütüphaneler Python paket yöneticisi olan pip aracılığıyla kurulmuştur. Örnek olarak veri işleme ve web geliştirme alanında yaygın olarak kullanılan bazı kütüphaneler aşağıdaki komut ile yüklenmiştir:

**pip install numpy pandas flask**

Projede kullanılan tüm bağımlılıkların kayıt altına alınması amacıyla aşağıdaki komut çalıştırılmıştır:

**pip freeze > requirements.txt**

Bu dosya sayesinde proje farklı bir bilgisayarda da aynı bağımlılıklar ile kolayca kurulabilmektedir.

**6. Kod Editörünün Yapılandırılması**

Kod geliştirme sürecini daha verimli hale getirmek amacıyla Visual Studio Code editörü yapılandırılmıştır. Editör içerisine çeşitli eklentiler yüklenerek geliştirme süreci desteklenmiştir.

Kurulan başlıca eklentiler şunlardır:

- Python Extension
- Pylance
- GitLens

Bu eklentiler sayesinde kod tamamlama, hata tespiti ve Git entegrasyonu gibi özellikler sağlanmıştır.

**7. Kodlama Standartlarının Belirlenmesi**

Yazılan kodların daha okunabilir ve sürdürülebilir olması için Python programlama dili için belirlenmiş olan **PEP 8** kodlama standardı uygulanmıştır.

Kod formatlama ve hata kontrolü için aşağıdaki araçlar kullanılmıştır:

**pip install black pylint**

Black aracı kodun otomatik olarak standart formata getirilmesini sağlarken, Pylint aracı kod içerisindeki hataları ve stil problemlerini tespit etmektedir.

**8. Versiyon Kontrol Sisteminin Yapılandırılması**

Proje dosyalarının sürüm kontrolünü sağlamak amacıyla Git versiyon kontrol sistemi kullanılmıştır. Git deposu aşağıdaki komut ile başlatılmıştır:

**git init**

Git kullanıcı bilgileri aşağıdaki komutlar ile sisteme tanımlanmıştır:

**git config --global user.name "Ad Soyad"**

**git config --global user.email "mail@example.com"**

**9. .gitignore Dosyasının Oluşturulması**

Projede gereksiz dosyaların versiyon kontrol sistemine dahil edilmesini engellemek amacıyla **.gitignore** dosyası oluşturulmuştur. Bu dosya içerisinde Git tarafından takip edilmemesi gereken dosya ve klasörler belirtilmektedir.

Örnek **.gitignore** içeriği aşağıdaki gibidir:

```
venv/
__pycache__/
.env
*.pyc
```

Bu sayede sanal ortam dosyaları ve geçici Python dosyaları Git deposuna eklenmemektedir.

**10. İlk Commit İşlemi**

Projenin başlangıç sürümünü kayıt altına almak amacıyla Git üzerinde ilk commit işlemi gerçekleştirilmiştir.

**git add .**

**git commit -m "Initial project setup"**

Bu işlem ile proje dosyalarının ilk sürümü Git deposuna kaydedilmiştir.